



خلاصه

در این آزمایش واحد کنترل دیجیتال یک سیستم انگوباتور طراحی و پیاده‌سازی شده است. طبق صورت آزمایش، حسگر، Heater و Cooler خارج از مدار قرار دارند؛ بنابراین دما به صورت یک عدد ۸ بیتی وارد مدار می‌شود و عملگرها با سیگنال‌های خروجی نمایش داده می‌شوند که روی برد FPGA به LED و Seven Segment وصل می‌شوند. واحد کنترل از دو ماشین حالت تشکیل شده است که دقیقاً از دو نمودار صورت آزمایش گرفته شده‌اند:

۱. ماشین اصلی که تصمیم می‌گیرد Heater و Cooler روشن باشند یا خاموش (نمودار سمت چپ).
 ۲. ماشین سرعت پنکه که در صورت روشن بودن Cooler، دور Fan را روی ۴، ۶ یا ۸ دور بر ثانیه تنظیم می‌کند (نمودار سمت راست).
- نکته‌ی مهم این است که ماشین دوم فقط وقتی فعال است که ماشین اول در حالت S_2 باشد؛ چون تنظیم دور پنکه‌ای که خاموش است بی‌معنی است.

نمایش دما و نرخ نمونه‌برداری

صورت آزمایش می‌گوید حسگر دمای بین -10 تا $+60$ درجه‌ی سانتیگراد را در قالب یک عدد ۸ بیتی تحویل می‌دهد و دما هر یک دقیقه یک بار خوانده می‌شود. برای نمایش دما از کد علامت‌دار مکمل ۲ استفاده شده است، یعنی خود عدد بر حسب درجه:

توضیح	مقدار	کد ۸ بیتی
سردترین مقدار قابل خواندن	-10	$8'hF6$
صفر درجه	0	$8'h00$
گرم‌ترین مقدار قابل خواندن	$+60$	$8'h3C$

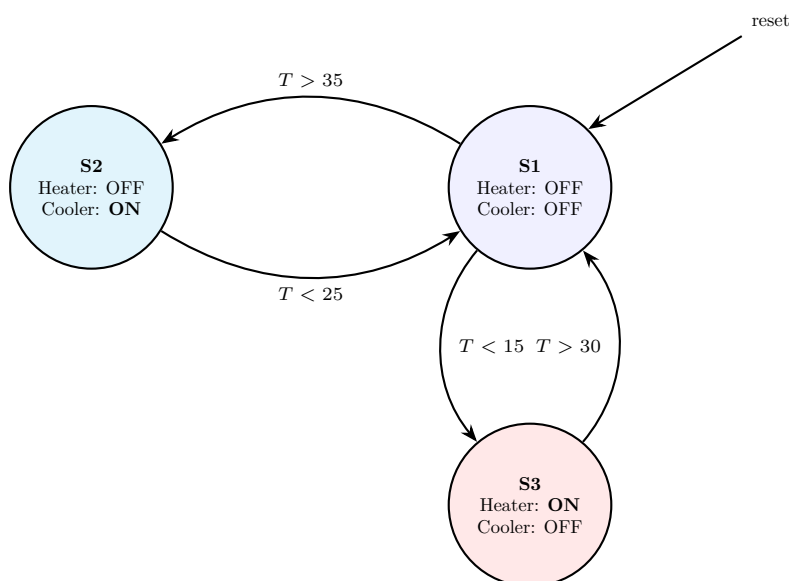
مزیت این انتخاب آن است که همه‌ی مقایسه‌ها دقیقاً همان‌طور نوشته می‌شوند که در نمودار حالت آمده‌اند؛ یعنی $temp > 35$ به جای مقایسه با یک کد میانی که باید جداگانه محاسبه شود.

برای نمونه‌برداری یک دقیقه‌ای، ماژول tick_gen یک پالس تک‌کلاکی به نام tick تولید می‌کند و هر دو ماشین حالت فقط با این پالس یک قدم جلو می‌روند. نسبت تقسیم به صورت پارامتر تعریف شده تا در سنتز مقدار واقعی و در شبیه‌سازی یک مقدار کوچک استفاده شود؛ در غیر این صورت تست‌بنچ باید چندین دقیقه زمان واقعی را شبیه‌سازی می‌کرد.

معماری کلی

ماژول	مسئولیت
main_fsm.v	ماشین حالت اصلی؛ خروجی‌های heater و cooler
fan_fsm.v	ماشین حالت سرعت پنکه؛ خروجی crs بر حسب RPS
tick_gen.v	تولید پالس نمونه‌برداری یک‌دقیقه‌ای
seven_seg_decoder.v	تبدیل عدد سرعت به کد هفت‌قطعه‌ای
incubator_controller.v	اتصال همه‌ی این‌ها به یکدیگر

ماشین حالت اصلی



نمودار فقط برای همین چهار گذار برچسب دارد. برای دمایی که هیچ‌کدام از شرط‌های خروجی حالت فعلی را برآورده نکند، ماشین در همان حالت می‌ماند. این دقیقاً چیزی است که به سیستم خاصیت پسماند (hysteresis) می‌دهد: یک باند مرده‌ی بین ۱۵ تا ۳۵ درجه که جلوی روشن و خاموش شدن پیاپی گرم‌کن و خنک‌کن حول یک نقطه‌ی تنظیم را می‌گیرد.

توجه کنید که همه‌ی مقایسه‌ها اکید هستند. مثلاً شرط گذار به S_2 عبارت است از $T > 35$ ، بنابراین در دمای دقیقاً ۳۵ درجه خنک‌کن روشن نمی‌شود. همین نکته در تست‌بنچ برای هر چهار آستانه جداگانه بررسی شده است.

```

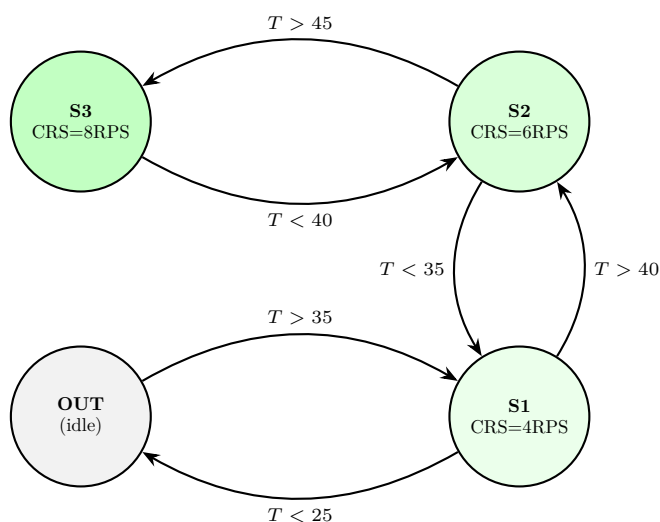
1 `include "incubator_pkg.vh"
2
3 module main_fsm (
4     input wire      Clk,
5     input wire      RstN,
6     input wire      tick,
7     input wire signed [7:0] temp,
8     output wire      heater,
9     output wire      cooler,
10    output wire [1:0] state,
11    output wire      cooler_next
12);
13    localparam [1:0] S1 = 2'd0,
14                  S2 = 2'd1,
15                  S3 = 2'd2;
16
17    reg [1:0] cur, nxt;
18
19    assign state = cur;
20    assign heater = (cur == S3);
21    assign cooler = (cur == S2);
  
```

```

22
23 assign cooler_next = (nxt == S2);
24
25 always @(*) begin
26     nxt = cur;
27     case (cur)
28     S1: begin
29         if (temp > `TH_COOL_ON)    nxt = S2;
30         else if (temp < `TH_HEAT_ON)  nxt = S3;
31     end
32     S2: if (temp < `TH_COOL_OFF)    nxt = S1;
33     S3: if (temp > `TH_HEAT_OFF)    nxt = S1;
34     default:                        nxt = S1;
35     endcase
36 end
37
38 always @(posedge Clk or negedge RstN) begin
39     if (!RstN)    cur <= S1;
40     else if (tick) cur <= nxt;
41 end
42 endmodule

```

ماشین حالت سرعت پمپ



طبق متن صورت آزمایش، این ماشین تنها زمانی فعال است که ماشین اصلی در حالت S_4 باشد و در غیر این صورت وارد حالت OUT خود می‌شود. بنابراین سیگنال enable که پایین می‌آید، حالت را مجبور به OUT می‌کند، نه اینکه صرفاً آن را منجمد نگه دارد. نتیجه این است که هر بار ورود دوباره به S_4 ، پمپ از پایین‌ترین پله‌ی نردبان سرعت شروع می‌کند، نه از سرعتی که قبلاً داشته است.

```

1 `include "incubator_pkg.vh"
2
3 module fan_fsm (
4     input wire          Clk,
5     input wire          RstN,
6     input wire          tick,
7     input wire signed [7:0] temp,
8     input wire          enable,
9
10    output wire [3:0]    crs,
11    output wire [1:0]    state
12 );
13 localparam [1:0] OUT  = 2'd0,
14                F_S1 = 2'd1,
15                F_S2 = 2'd2,
16                F_S3 = 2'd3;
17
18 reg [1:0] cur, nxt;
19
20 assign state = cur;
21
22 assign crs = (cur == F_S1) ? `CRS_4 :
23             (cur == F_S2) ? `CRS_6 :
24             (cur == F_S3) ? `CRS_8 : `CRS_OFF;

```

```

25
26 always @(*) begin
27     nxt = cur;
28     case (cur)
29         OUT: if (temp > `TH_FAN_IN)    nxt = F_S1;
30         F_S1: begin
31             if (temp > `TH_FAN_UP1) nxt = F_S2;
32             else if (temp < `TH_FAN_OUT) nxt = OUT;
33         end
34         F_S2: begin
35             if (temp > `TH_FAN_UP2)    nxt = F_S3;
36             else if (temp < `TH_FAN_DOWN1) nxt = F_S1;
37         end
38         F_S3: if (temp < `TH_FAN_DOWN2)  nxt = F_S2;
39         default:                        nxt = OUT;
40     endcase
41 end
42
43 always @(posedge Clk or negedge RstN) begin
44     if (!RstN) cur <= OUT;
45     else if (!enable) cur <= OUT;
46     else if (tick) cur <= nxt;
47 end
48 endmodule

```

نکته‌ی طراحی: هم‌زمانی دو ماشین حالت

مهم‌ترین نکته‌ی ظریف این آزمایش، لحظه‌ی دقیق فعال یا غیرفعال شدن ماشین دوم است.

اگر سیگنال enable را مستقیماً به خروجی cooler (یعنی حالت فعلی ماشین اصلی) وصل کنیم، در همان tick که ماشین اصلی از S_2 خارج می‌شود، مقدار cooler هنوز مقدار قدیمی است. در نتیجه ماشین پنکه یک قدم اضافه برمی‌دارد و برای یک دوره‌ی کامل نمونه‌برداری (یعنی یک دقیقه‌ی واقعی) وضعیتی نمایش داده می‌شود که در آن خنک‌کن خاموش است ولی پنکه هنوز در حال چرخش است. این دقیقاً همان چیزی است که متن آزمایش می‌گوید بی‌معنی است.

راه‌حل این است که ماشین پنکه با حالت بعدی ماشین اصلی گیت شود:

```

// in main_fsm: the state this FSM will hold AFTER the current tick
assign cooler_next = (nxt == S2);

// in incubator_controller: gate the fan on the next state, not the current one
fan_fsm u_fan ( ... .enable(cooler_next), ... );

```

با این تغییر، همان tick که خنک‌کن را خاموش می‌کند، پنکه را هم در OUT پارک می‌کند و هر دو ماشین بر اساس یک نمونه‌ی دما به‌صورت سازگار عمل می‌کنند. در سمت ورود هم رفتار درست است: چون شرط $T > 35$ باعث ورود ماشین اصلی به S_2 می‌شود و هم شرط گذار OUT به S_1 در ماشین پنکه است، خنک‌کن در همان لحظه‌ی روشن شدن سرعت اولیه‌ی ۴ RPS خود را می‌گیرد.

اتصال ماژول‌ها و خروجی‌های برد

طبق تأکید صورت آزمایش، به جای Heater و Cooler واقعی، خروجی‌ها با LED و Seven Segment نمایش داده می‌شوند. بنابراین خروجی crs علاوه بر مقدار عددی، به‌صورت کد هفت‌قطعه‌ای (crs_seg) هم بیرون داده می‌شود تا طراحی بدون منطق واسط اضافه روی برد قابل استفاده باشد.

سیگنال	جهت	پهنای	توضیح
Clk	ورودی	۱	کلاک برد
RstN	ورودی	۱	ریست ناهم‌گام فعال پایین
temp	ورودی	۸	دمای حسگر، علامت‌دار، ۱۰- تا ۶۰+
heater	خروجی	۱	LED گرم‌کن
cooler	خروجی	۱	LED خنک‌کن
crs	خروجی	۴	دور پیکه بر حسب RPS (۰/۴/۶/۸)
crs_seg	خروجی	۷	همان مقدار، کدشده برای Seven Segment

```

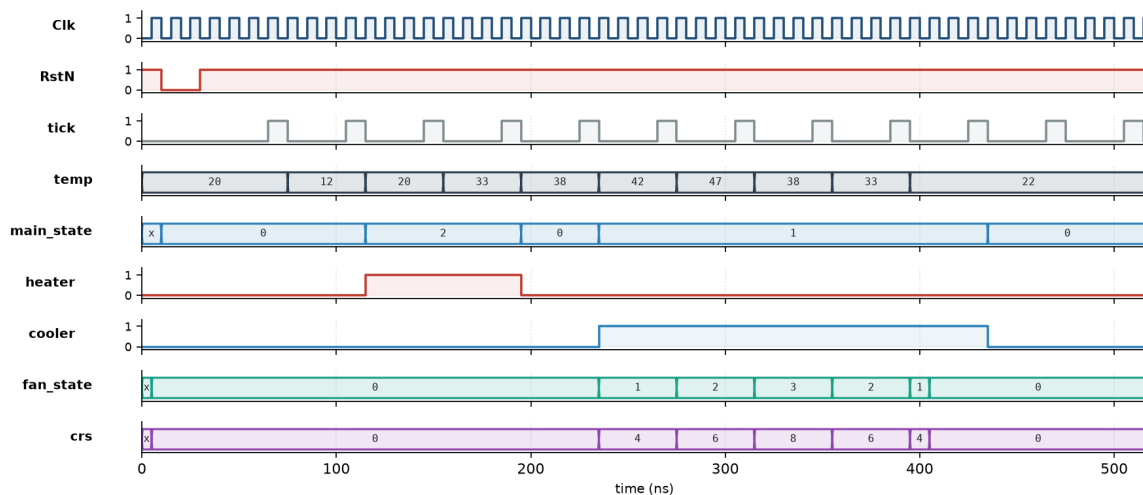
1  `include "incubator_pkg.vh"
2
3  module incubator_controller #(
4      parameter integer TICK_DIVIDE = 60_000_000
5  ) (
6      input wire          Clk,
7      input wire          RstN,
8      input wire signed [7:0] temp,
9
10     output wire          heater,
11     output wire          cooler,
12     output wire [3:0]    crs,
13     output wire [6:0]    crs_seg,
14     output wire [1:0]    main_state,
15     output wire [1:0]    fan_state,
16     output wire          tick
17 );
18     wire          cooler_next;
19     wire [1:0]    m_state;
20
21     tick_gen #(.DIVIDE(TICK_DIVIDE)) u_tick (
22         .Clk(Clk), .RstN(RstN), .tick(tick)
23     );
24
25     main_fsm u_main (
26         .Clk      (Clk),
27         .RstN     (RstN),
28         .tick     (tick),
29         .temp     (temp),
30         .heater    (heater),
31         .cooler   (cooler),
32         .state    (m_state),
33         .cooler_next (cooler_next)
34     );
35
36     fan_fsm u_fan (
37         .Clk      (Clk),
38         .RstN     (RstN),
39         .tick     (tick),
40         .temp     (temp),
41         .enable   (cooler_next),
42         .crs      (crs),
43         .state    (fan_state)
44     );
45
46     assign main_state = m_state;
47
48     seven_seg_decoder u_seg (.value(crs), .seg(crs_seg));
49 endmodule

```

شکل موج

شکل موج یک چرخه‌ی کامل حرارتی را نشان می‌دهد: از باند مرده شروع می‌شود، به سرما می‌رود و گرم‌کن روشن می‌شود، دوباره گرم می‌شود تا خنک‌کن روشن شود و پیکه نردبان ۸ → ۶ → ۴ را بالا و پایین برود، و در پایان همه‌چیز خاموش می‌شود.

Incubator control unit - full thermal cycle: heating, dead band, cooling with fan 4/6/8 RPS



نکات قابل مشاهده: در دمای ۱۲ درجه گرم‌کن روشن می‌شود و در ۲۰ درجه همچنان روشن می‌ماند — این همان خاصیت پسماند است، نه اشکال. در ۳۳ درجه گرم‌کن خاموش می‌شود و در ۳۸ درجه خنک‌کن روشن شده و پنکه بلافاصله روی ۴ RPS قرار می‌گیرد. سپس با ۴۲ و ۴۷ درجه به ۶ و ۸ می‌رسد و در مسیر برگشت دوباره پایین می‌آید. در پایان با رسیدن دما به ۲۲ درجه، cooler پایین می‌آید و در همان لحظه مقدار crs صفر می‌شود؛ یعنی همان رفتاری که در بخش قبل درباره‌اش صحبت شد.

تست بنچ اول: مقایسه با مدل مرجع

تست بنچ اصلی یک مدل مرجع مستقل از هر دو نمودار حالت نگه می‌دارد و بعد از هر نمونه‌برداری، حالت هر دو ماشین به همراه خروجی‌های heater، cooler و crs را با آن مقایسه می‌کند.

پوشش تست شامل این موارد است: ریست، باند مرده‌ی ۱۵ تا ۳۵ درجه، چرخه‌ی گرمایش، چرخه‌ی سرمایش با کل نردبان پنکه، ریست وسط کار، و مهم‌تر از همه یک جاروب کامل: از چهار پیکربندی مختلف (main, fan) شروع می‌شود و از هر کدام تمام دماهای بازه‌ی ۱۰- تا ۶۰+ اعمال و با مدل مقایسه می‌شود.

علاوه بر این، یک invariant به صورت پیوسته بررسی می‌شود:

```
always @(posedge Clk)
  if (RstN && !cooler && crs != 4'd0) ... // FAIL
```

این بررسی در هر لبه‌ی کلاک انجام می‌شود، نه فقط در نقاط نمونه‌برداری؛ بنابراین حتی یک لحظه چرخش پنکه در زمان خاموشی خنک‌کن هم تست را رد می‌کند.

```
1 `timescale 1ns/1ps
2 `include "incubator_pkg.vh"
3
4 module tb_incubator;
5     localparam integer TICKDIV = 4;
6     reg Clk, RstN;
7     reg signed [7:0] temp;
8     wire heater, cooler, tick;
9     wire [3:0] crs;
10    wire [6:0] crs_seg;
11    wire [1:0] main_state, fan_state;
12
13    incubator_controller #(.TICK_DIVIDE(TICKDIV)) dut (
14        .Clk(Clk), .RstN(RstN), .temp(temp),
15        .heater(heater), .cooler(cooler), .crs(crs), .crs_seg(crs_seg),
16        .main_state(main_state), .fan_state(fan_state), .tick(tick)
17    );
18
19    initial Clk = 1'b0;
20    always #5 Clk = ~Clk;
```

```

21 integer pass, fail;
22 integer t, k;
23
24
25 localparam [1:0] R_S1 = 2'd0, R_S2 = 2'd1, R_S3 = 2'd2;
26 localparam [1:0] R_OUT = 2'd0, R_F1 = 2'd1, R_F2 = 2'd2, R_F3 = 2'd3;
27
28 reg [1:0] ref_main, ref_fan;
29
30 task automatic ref_reset;
31 begin
32     ref_main = R_S1;
33     ref_fan = R_OUT;
34 end
35 endtask
36
37 task automatic ref_step;
38 input signed [7:0] T;
39 reg [1:0] nm;
40 begin
41     nm = ref_main;
42     case (ref_main)
43         R_S1: begin
44             if (T > 35) nm = R_S2;
45             else if (T < 15) nm = R_S3;
46         end
47         R_S2: if (T < 25) nm = R_S1;
48         R_S3: if (T > 30) nm = R_S1;
49     endcase
50
51     if (nm != R_S2) begin
52         ref_fan = R_OUT;
53     end else begin
54         case (ref_fan)
55             R_OUT: if (T > 35) ref_fan = R_F1;
56             R_F1: begin
57                 if (T > 40) ref_fan = R_F2;
58                 else if (T < 25) ref_fan = R_OUT;
59             end
60             R_F2: begin
61                 if (T > 45) ref_fan = R_F3;
62                 else if (T < 35) ref_fan = R_F1;
63             end
64             R_F3: if (T < 40) ref_fan = R_F2;
65         endcase
66     end
67
68     ref_main = nm;
69 end
70 endtask
71
72 function [3:0] ref_crs;
73 input [1:0] fs;
74 begin
75     case (fs)
76         R_F1: ref_crs = 4'd4;
77         R_F2: ref_crs = 4'd6;
78         R_F3: ref_crs = 4'd8;
79         default: ref_crs = 4'd0;
80     endcase
81 end
82 endfunction
83
84 task automatic check;
85 input [255:0] tag;
86 reg exp_heater, exp_cooler;
87 begin
88     exp_heater = (ref_main == R_S3);
89     exp_cooler = (ref_main == R_S2);
90     if (main_state == ref_main && fan_state == ref_fan &&
91         heater == exp_heater && cooler == exp_cooler &&
92         crs == ref_crs(ref_fan)) begin
93         pass = pass + 1;
94     end else begin
95         $display("FAIL [%0s] T=%0d | main got=%0d exp=%0d | fan got=%0d exp=%0d | H=%b/%b C=%b/%b
96         crs=%0d/%0d",
97             tag, temp, main_state, ref_main, fan_state, ref_fan,
98             heater, exp_heater, cooler, exp_cooler,
99             crs, ref_crs(ref_fan));
100         fail = fail + 1;
101     end
102 end
103 endtask
104
105 task automatic sample;
106 input signed [7:0] T;
107 input [255:0] tag;

```

```

107     begin
108         temp = T;
109         @(posedge tick);
110         @(posedge Clk);
111         #1;
112         ref_step(T);
113         check(tag);
114     end
115 endtask
116
117 task automatic do_reset;
118     begin
119         @(negedge Clk);
120         RstN = 1'b0;
121         temp = 8'sd20;
122         @(negedge Clk);
123         @(negedge Clk);
124         RstN = 1'b1;
125         ref_reset;
126         #1;
127         check("reset");
128     end
129 endtask
130
131 initial begin
132     pass = 0; fail = 0;
133     RstN = 1'b1; temp = 8'sd20;
134     ref_reset;
135
136     do_reset;
137
138     if (heater === 1'b0 && cooler === 1'b0 && crs === 4'd0 &&
139         main_state === R_S1 && fan_state === R_OUT)
140         pass = pass + 1;
141     else begin
142         $display("FAIL: wrong reset outputs");
143         fail = fail + 1;
144     end
145
146     for (t = 15; t <= 35; t = t + 1)
147         sample(t[7:0], "deadband_hold");
148
149     sample(8'sd14, "heat_on");
150     sample(-8'sd10, "heat_stay_min");
151     sample(8'sd20, "heat_hold");
152     sample(8'sd30, "heat_hold_edge");
153     sample(8'sd31, "heat_off");
154
155     sample(8'sd35, "cool_edge_hold");
156     sample(8'sd36, "cool_on");
157     sample(8'sd36, "fan_out_to_s1");
158     sample(8'sd41, "fan_s1_to_s2");
159     sample(8'sd46, "fan_s2_to_s3");
160     sample(8'sd60, "fan_max_temp");
161     sample(8'sd39, "fan_s3_to_s2");
162     sample(8'sd34, "fan_s2_to_s1");
163     sample(8'sd30, "fan_s1_hold");
164     sample(8'sd24, "cool_off");
165
166     sample(8'sd36, "reenter_cool");
167     sample(8'sd46, "reenter_fan1");
168     sample(8'sd46, "reenter_fan2");
169     sample(8'sd46, "reenter_fan3");
170     sample(8'sd10, "cool_off_again");
171
172     sample(8'sd50, "pre_reset_cool");
173     sample(8'sd50, "pre_reset_fan");
174     do_reset;
175
176     for (k = 0; k < 4; k = k + 1) begin
177         do_reset;
178         case (k)
179             0: ;
180             1: begin sample(8'sd14, "setup"); end
181             2: begin sample(8'sd36, "setup");
182                  sample(8'sd36, "setup"); end
183             3: begin sample(8'sd36, "setup");
184                  sample(8'sd36, "setup");
185                  sample(8'sd41, "setup");
186                  sample(8'sd46, "setup"); end
187         endcase
188         for (t = -10; t <= 60; t = t + 1)
189             sample(t[7:0], "sweep");
190     end
191
192     do_reset;
193     sample(8'sd36, "seg_setup");

```

```

194 sample(8'sd36, "seg_4rps");
195 if (crs === 4'd4 && crs_seg === 7'b0011001) pass = pass + 1;
196 else begin
197     $display("FAIL: seg for 4 RPS = %b", crs_seg);
198     fail = fail + 1;
199 end
200
201 $display("tb_incubator: Passed: %0d, Failed: %0d", pass, fail);
202 if (fail != 0) $fatal(1);
203 $finish;
204 end
205
206 always @(posedge Clk) begin
207     if (RstN && !cooler && crs != 4'd0) begin
208         $display("FAIL: crs=%0d while cooler is OFF (T=%0d, fan_state=%0d)",
209             crs, temp, fan_state);
210         fail = fail + 1;
211     end
212 end
213
214 initial begin
215     #2000000;
216     $display("FAIL: timeout");
217     $fatal(1);
218 end
219 endmodule

```

تست‌بنچ دوم: انطباق با نمودار

مدل مرجع بررسی قدرتمندی است، اما یک ضعف ذاتی دارد: مدلی که به دست همان طراح نوشته شده، ممکن است همان برداشت اشتباه از صورت مسئله را تکرار کند. دقیقاً همین اتفاق در این آزمایش افتاد — هم RTL و هم مدل مرجع، در ابتدا اجازه می‌دادند پیکه در لحظه‌ی خاموش شدن خنک‌کن یک قدم اضافه بردارد و هر دو با هم موافق بودند، پس تست سبز بود.

به همین دلیل یک تست‌بنچ دوم نوشته شد که مستقیماً از روی نمودارهای صورت آزمایش و بدون هیچ مدل مشترکی، فهرستی از رفتارهایی را که باید برقرار باشند بررسی می‌کند: مقایسه‌های اکید روی هر شش آستانه، پله‌های ۴/۶/۸، صفر بودن سرعت در تمام حالت‌هایی که خنک‌کن خاموش است، و شروع دوباره از پایین نردبان پس از بازگشت به S_4 . همین تست بود که اشکال را پیدا کرد.

```

1 `timescale 1ns/1ps
2
3 module tb_incubator_spec;
4     localparam integer TD = 4;
5     reg Clk=0, RstN=1; reg signed [7:0] temp=8'sd20;
6     wire heater,cooler,tick; wire [3:0] crs; wire [6:0] seg;
7     wire [1:0] ms, fs;
8     incubator_controller #(TICK_DIVIDE(TD)) dut(.Clk(Clk),.RstN(RstN),.temp(temp),
9         .heater(heater),.cooler(cooler),.crs(crs),.crs_seg(seg),
10        .main_state(ms),.fan_state(fs),.tick(tick));
11     always #5 Clk=~Clk;
12     integer bad=0, checks=0;
13     task automatic step(input signed [7:0] T);
14     begin temp=T; @(posedge tick); @(posedge Clk); #1; end endtask
15     task automatic want(input [255:0] w, input ok);
16     begin checks=checks+1; if(!ok) begin $display(" MISMATCH: %0s (T=%0d ms=%0d fs=%0d H=%b C=%b crs=%0d)",w
17         ,temp,ms,fs,heater,cooler,crs); bad=bad+1; end end endtask
18     task automatic rst; begin @(negedge Clk); RstN=0; @(negedge Clk); @(negedge Clk); RstN=1; #1; end endtask
19     initial begin
20         rst;
21         $display("A) reset => S1, all off");
22         want("reset S1 off", ms==0 && !heater && !cooler && crs==0);
23
24         $display("B) strict inequalities on the boundaries");
25         rst; step(8'sd35); want("T=35 must NOT turn cooler on", !cooler);
26         rst; step(8'sd36); want("T=36 turns cooler on", cooler);
27         rst; step(8'sd15); want("T=15 must NOT turn heater on", !heater);
28         rst; step(8'sd14); want("T=14 turns heater on", heater);
29         rst; step(8'sd14); step(8'sd30); want("T=30 keeps heater on", heater);
30         rst; step(8'sd14); step(8'sd31); want("T=31 turns heater off", !heater);
31         rst; step(8'sd36); step(8'sd25); want("T=25 keeps cooler on", cooler);
32         rst; step(8'sd36); step(8'sd24); want("T=24 turns cooler off", !cooler);
33
34         $display("C) fan ladder speeds are 4/6/8 RPS");
35         rst; step(8'sd36); step(8'sd36); want("fan 4RPS", crs==4);
36         step(8'sd41); want("fan 6RPS", crs==6);

```

```

36 step(8'sd46); want("fan 8RPS", crs==8);
37 step(8'sd39); want("back to 6RPS", crs==6);
38 step(8'sd34); want("back to 4RPS", crs==4);
39
40 $display("D) fan is meaningless while cooler is off => crs must be 0");
41 rst; want("crs=0 in S1", crs==0);
42 rst; step(8'sd14); want("crs=0 while heating (S3)", crs==0 && heater);
43 rst; step(8'sd60); step(8'sd60); step(8'sd60);
44 want("hot => cooler on, fan running", cooler && crs>0);
45 step(8'sd10);
46 want("leaving S2 parks fan at 0", !cooler && crs==0 && fs==0);
47
48 $display("E) re-entering S2 restarts the fan from OUT (not from 8RPS)");
49 step(8'sd60);
50 want("re-entry: fan restarts at 4RPS (not 8)", cooler && crs==4);
51 step(8'sd60);
52 want("re-entry: then climbs to 6RPS", crs==6);
53
54 $display("F) hold behaviour inside the dead band");
55 rst; step(8'sd20); want("T=20 from reset stays S1", ms==0 && !heater && !cooler);
56 rst; step(8'sd14); step(8'sd20); want("T=20 while heating keeps heating", heater);
57 rst; step(8'sd36); step(8'sd30); want("T=30 while cooling keeps cooling", cooler);
58
59 $display("tb_incubator_spec: Passed: %0d, Failed: %0d", checks - bad, bad);
60 if (bad != 0) $fatal(1);
61 $finish;
62 end
63 endmodule

```

نتایج تست

```

=== tb_incubator ===
tb_incubator: Passed: 345, Failed: 0
PASS: tb_incubator

=== tb_incubator_spec ===
tb_incubator_spec: Passed: 23, Failed: 0
PASS: tb_incubator_spec

summary: 2/2 passed, 0 failed

```

جمع‌بندی

در این آزمایش دو ماشین حالت طراحی شد که یکی از آن‌ها مشروط به وضعیت دیگری فعال می‌شود. تجربه‌ی اصلی این بود که وقتی دو ماشین حالت روی یک رویداد مشترک (اینجا پالس نمونه‌برداری) کار می‌کنند، باید دقیقاً مشخص کرد که هر کدام بر اساس کدام نسخه از حالت دیگری تصمیم می‌گیرد؛ حالت فعلی یا حالت بعدی. انتخاب اشتباه، مداری می‌سازد که نه در شبیه‌سازی خطای نحوی می‌دهد و نه لزوماً در تست‌های ساده دیده می‌شود، ولی رفتار فیزیکی نادرستی تولید می‌کند.

در نهایت، خاصیت پسماند در این طراحی یک ویژگی عمدی است نه اتفاقی. باند مرده‌ی ۱۵ تا ۳۵ درجه باعث می‌شود سیستم حول یک نقطه‌ی تنظیم دچار روشن و خاموش شدن پیایی نشود، که هم برای عمر عملگرها و هم برای پایداری دمای داخل محفظه مطلوب است.

خروجی سنتز (Quartus Prime)

طرح روی Intel Quartus Prime Lite Edition 21.1.0 با موجودیت سطح بالای incubator_controller (شامل fan_fsm، main_fsm، tick_gen و seven_seg_decoder) سنتز و Route & Place شد. دستگاه هدف: Cyclone V (5CGXFC7C7F23C8).

مورد	نتیجه
وضعیت کامپایل	موفق (۰ خطا، ۱۷ هشدار)
Logic utilization (ALMs)	۴۴/۵۶۴۸۰ (< ۱٪)
Total registers	۴۱
Total pins	۲۸/۲۶۸ (۱۰٪)
Block memory bits	۰
DSP Blocks	۰

